Day 12 | Task 3

Creation and insertion of data:

```
❯ sqlite3 d_12_t3.db
SQLite version 3.54.0 2026-10-09 15:46:58
Enter ".help" for usage hints.
SQLite-3.54 d_12_t3.db-> create table salesorders (orderid text primary key, product text, category te
SQLite-3.54 d_12_t3.db-> insert into salesorders values ('ORD101', 'Running Shoes', 'Footwear', 'benga
', '1', '5999', '2026-10-02'), ('ORD104', 'Cotton T-Shirt', 'Clothing', 'Bengaluru ', '3', '499', '202
-05'), ('ORD107', 'Sunglasses', 'Accessories', NULL, '2', '-150', 'invalid-date'), ('ORD108', 'Hoodie'
sories', 'Delhi', '1', '9999', '2026-10-08');
SQLite-3.54 d_12_t3.db-> select * from salesorders
                    ;-> ;
```

orderid	product	category	city	quantity	unitprice	orderdate
ORD101	Running Shoes	Footwear	bengaluru	'2'	'2499'	2026-10-01
ORD102	Denim Jacket	Clothing	CHENNAI	'1'	'1899'	2026-10-02
ORD103	Smart Watch	Electronics	Hyderabad	'1'	'5999'	2026-10-02
ORD104	Cotton T-Shirt	Clothing	'Bengaluru '	'3'	'499'	2026-10-03
ORD105	Leather Bag	Accessories	Mumbai	two	'3499'	2026-10-04
ORD106	Running Shoes	Footwear	Chennai	'1'	'2499'	2026-10-05
ORD107	Sunglasses	Accessories	NULL	'2'	'-150'	invalid-date
ORD108	Hoodie	Clothing	MUMBAI	'2'	'1299'	2026-10-06
ORD109	Smart Watch	Electronics	Hyderabad	'1'	NULL	2026-10-07
ORD110	Leather Bag	Accessories	Delhi	'1'	'9999'	2026-10-08

1. Find all unique product categories

None

```
select distinct category from salesorders;
```

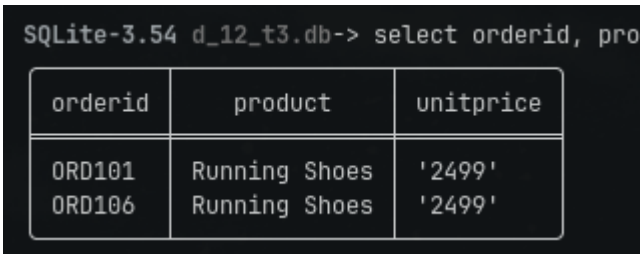
```
SQLite-3.54 d_12_t3.db->
```

category
Footwear
Clothing
Electronics
Accessories

2. Find products containing the word "shoes"

None

```
select orderid, product, unitprice from
salesorders where product like '%shoes%';
```



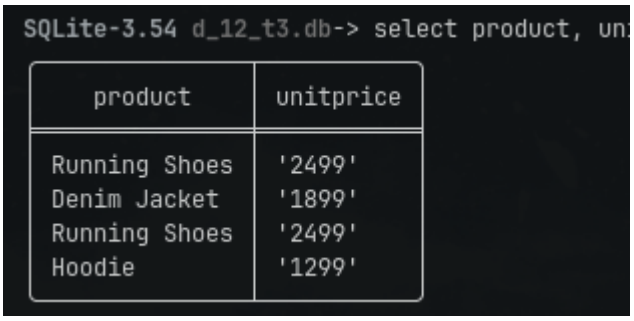
```
SQLite-3.54 d_12_t3.db-> select orderid, pro
```

orderid	product	unitprice
ORD101	Running Shoes	'2499'
ORD106	Running Shoes	'2499'

3. Find products priced between 1,000 and 3,000

None

```
select product, unitprice from salesorders
where cast(unitprice as integer) between 1000
and 3000;
```



```
SQLite-3.54 d_12_t3.db-> select product, un
```

product	unitprice
Running Shoes	'2499'
Denim Jacket	'1899'
Running Shoes	'2499'
Hoodie	'1299'

4. Find orders with missing city or price information

None

```
select * from salesorders where city is null or
unitprice is null;
```

```
SQLite-3.54 d_12_t3.db-> select * from salesorders where city is null or unitprice is null;
```

orderid	product	category	city	quantity	unitprice	orderdate
ORD107	Sunglasses	Accessories	NULL	'2'	'-150'	invalid-date
ORD109	Smart Watch	Electronics	Hyderabad	'1'	NULL	2026-10-07

5. Find orders from selected cities

None

```
select orderid, product, city from salesorders
where lower(trim(city)) in ('chennai',
'mumbai', 'hyderabad');
```

```
SQLite-3.54 d_12_t3.db-> select orderid, product,
```

orderid	product	city
ORD102	Denim Jacket	CHENNAI
ORD103	Smart Watch	Hyderabad
ORD105	Leather Bag	Mumbai
ORD106	Running Shoes	Chennai
ORD108	Hoodie	MUMBAI
ORD109	Smart Watch	Hyderabad

6. Display the five most expensive orders

None

```
select orderid, product, unitprice from
salesorders where cast(unitprice as integer) >
0 order by cast(unitprice as integer) desc
limit 5;
```

```
SQLite-3.54 d_12_t3.db-> select orderid, prod
```

orderid	product	unitprice
ORD110	Leather Bag	'9999'
ORD103	Smart Watch	'5999'
ORD105	Leather Bag	'3499'
ORD101	Running Shoes	'2499'
ORD106	Running Shoes	'2499'

7. Categorize orders by price

None

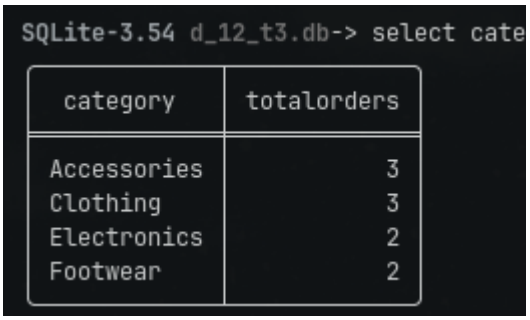
```
select orderid, product, case when
cast(unitprice as integer) < 0 then 'invalid
price' when unitprice is null then 'missing
price' when cast(unitprice as integer) < 1500
then 'low price' when cast(unitprice as
integer) <= 3500 then 'medium price' else 'high
price' end as pricecategory from salesorders;
```

```
SQLite-3.54 d_12_t3.db-> select orderid, product, case
rice' else 'high price' end as pricecategory from sale
```

orderid	product	pricecategory
ORD101	Running Shoes	medium price
ORD102	Denim Jacket	medium price
ORD103	Smart Watch	high price
ORD104	Cotton T-Shirt	low price
ORD105	Leather Bag	medium price
ORD106	Running Shoes	medium price
ORD107	Sunglasses	invalid price
ORD108	Hoodie	low price
ORD109	Smart Watch	missing price
ORD110	Leather Bag	high price

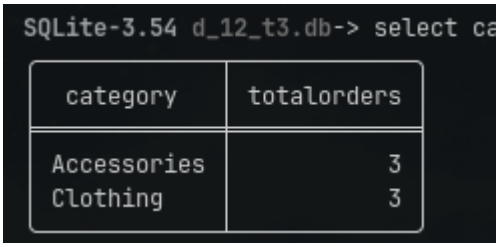
8. Count orders by category

```
None
select category, count(*) as totalorders from
salesorders group by category;
```



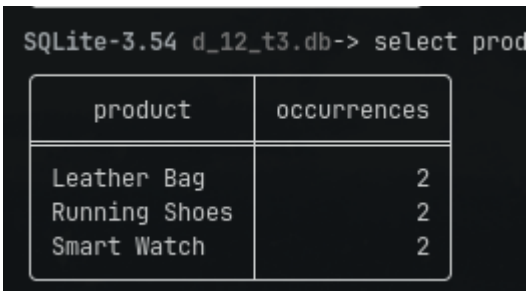
9. Find categories with more than two orders

```
None
select category, count(*) as totalorders from
salesorders group by category having count(*) >
2;
```



10. Find duplicate product names

```
None
select product, count(*) as occurrences from
salesorders group by product having count(*) >
1;
```


SQLite-3.54 d_12_t3.db-> select prod
product
occurrences
Leather Bag
2
Running Shoes
2
Smart Watch
2